

MINI PROJECT: Linux Server Maintenance Automation Tool

INTRODUCTION:

This mini project is designed to learn how to automate common Linux server administration tasks using Shell Scripting.

It covers real-world DevOps tasks such as creating users, checking services, backing up data, analyzing logs, cleaning old logs, and generating configuration files.

This project helps to understand:

- Loops (for, while)
- Functions and return codes
- File handling (read, write, heredoc)
- Case and select statements
- System commands (tar, cut, sort, uniq, head, tail)
- Automation logic and real-time DevOps scenarios

PROJECT OBJECTIVE:

To build a menu-driven automation tool that performs:

1. User creation from a file
2. Service health check and auto-restart
3. Backup of important directories
4. Log analysis with cut/sort/uniq/head/tail
5. Automatic cleanup of old logs
6. Configuration file creation using heredoc

Project Structure:

server-maintenance/

|

|— server-maintenance.sh

|— users.txt # List of users to create

```
|— logs/           # Log folder
|  |— actions.log
|— backups/       # Backup folder
```

Script (server-maintenance.sh)

```
#!/bin/bash
```

```
LOGFILE="logs/actions.log"
```

```
BACKUP_DIR="backups"
```

```
USER_FILE="users.txt"
```

```
mkdir -p logs
```

```
mkdir -p backups
```

```
# -----
```

```
# 1. Function: Create users from a file
```

```
# -----
```

```
create_users() {
```

```
    echo "----- Creating Users -----" | tee -a $LOGFILE
```

```
    while read user
```

```
    do
```

```
        [ -z "$user" ] && continue
```

```
        useradd "$user" && \
```

```
            echo "Created user: $user" | tee -a $LOGFILE || \
```

```
            echo "Failed to create: $user" | tee -a $LOGFILE
```

```

done < "$USER_FILE"
}

# -----
# 2. Function: Check and restart service
# -----

check_service() {
    read -p "Enter service name (httpd/nginx/mysql): " SERVICE

    if systemctl is-active --quiet "$SERVICE"; then
        echo "$SERVICE is running." | tee -a $LOGFILE
    else
        echo "$SERVICE is NOT running. Restarting..." | tee -a $LOGFILE
        systemctl restart "$SERVICE"

        if systemctl is-active --quiet "$SERVICE"; then
            echo "$SERVICE restarted successfully." | tee -a $LOGFILE
        else
            echo "Failed to restart $SERVICE." | tee -a $LOGFILE
        fi
    fi
}

# -----
# 3. Function: Backup a directory
# -----

backup_data() {
    read -p "Enter directory to backup: " SOURCE

```

```
TS=$(date +"%Y-%m-%d_%H-%M-%S")
```

```
BACKUP_FILE="$BACKUP_DIR/backup_$TS.tar.gz"
```

```
tar -czf "$BACKUP_FILE" "$SOURCE"
```

```
if [ $? -eq 0 ]; then
```

```
    echo "Backup created: $BACKUP_FILE" | tee -a $LOGFILE
```

```
else
```

```
    echo "Backup FAILED" | tee -a $LOGFILE
```

```
fi
```

```
}
```

```
# -----
```

```
# 4. Function: Analyze system logs
```

```
# -----
```

```
analyze_logs() {
```

```
    echo "----- Last 10 login events ----"
```

```
    last | head
```

```
    echo ""
```

```
    echo "----- Unique users who logged in ----"
```

```
    last | cut -d " " -f1 | sort | uniq
```

```
}
```

```
# -----
```

```
# 5. Function: Cleanup old logs
```

```
# -----
```

```

clean_logs() {
    find logs/ -type f -mtime +7 -delete
    return $?
}

# -----
# 6. Function: Create config file using heredoc
# -----

create_config() {
    cat <<EOF > /tmp/myapp.conf
server {
    listen 80;

    server_name example.com;

    location / {
        proxy_pass http://localhost:5000;
    }
}
EOF

    echo "Config file created at: /tmp/myapp.conf" | tee -a $LOGFILE
}

# -----
# 7. MENU (select + case)
# -----

PS3="Choose an option: "

```

```
select option in \  
    "Create Users" \  
    "Check/Restart Service" \  
    "Backup Directory" \  
    "Analyze Logs" \  
    "Clean Old Logs" \  
    "Create Config File" \  
    "Exit"  
do  
    case $option in  
  
        "Create Users")  
            create_users  
            ;;  
  
        "Check/Restart Service")  
            check_service  
            ;;  
  
        "Backup Directory")  
            backup_data  
            ;;  
  
        "Analyze Logs")  
            analyze_logs  
            ;;  
  
        "Clean Old Logs")
```

```
clean_logs
if [ $? -eq 0 ]; then
    echo "Old logs cleaned successfully." | tee -a $LOGFILE
else
    echo "Log cleanup failed." | tee -a $LOGFILE
fi
;;

"Create Config File")
    create_config
;;

"Exit")
    echo "Goodbye!"
    exit
;;

*) echo "Invalid option" ;;

esac

done
```